06 - Testing & Validación

Estrategia de Testing

```
Unit Tests
├─ Unity CLI wrapper
├─ Blender CLI wrapper
└─ Input validation

Integration Tests
├─ MCP Server ↔ Claude Code
├─ Unity Project operations
└─ Blender operations

End-to-End Tests
├─ Create script workflow
├─ Create model + export workflow
└─ Full game asset generation
```

Testing Manual (Sin Framework)

Test 1: Unity MCP - CreateScript

```
# Terminal 1: Iniciar servidor
cd MCP/servers/unity
AUTO_COMPILE=false npm start

# Terminal 2: Hacer petición
node -e "
const { exec } = require('child_process');
const msg = {
  jsonrpc: '2.0',
  id: 1,
  method: 'tools/call',
  params: {
    name: 'create_script',
    arguments: {
      path: 'Assets/Scripts/Test.cs',
      content: 'public class Test { public void Greet() { Debug.Log(\"Hello\"); }
}',
      namespace: 'ZoremGame.Test'
    }
  }
};
```

```
console.log(JSON.stringify(msg));
"
```

Resultado esperado:

- Archivo creado en `Assets/Scripts/Test.cs`
- Contiene namespace correcto
- Sin errores de compilación

Test 2: Blender MCP - CreateModel

```
# Terminal 1: Iniciar servidor
cd MCP/servers/blender
npm start

# Terminal 2: Hacer petición (similar a Test 1)
# Pero con parámetros: create_model, type: "humanoid", name: "TestBot"
```

Resultado esperado:

- Archivo .blend creado en output path
- Contiene modelo humanoid
- Pronto a exportar

Test 3: End-to-End en Claude Code

En Claude Code, ejecuta estos prompts en orden:

Test 3a: Crear 2 scripts

```
Crea dos scripts en Assets/Scripts/Gameplay/:
1. PlayerInput.cs - captura input WASD
2. PlayerMovement.cs - aplica movimiento basado en input
```

Resultado: 2 archivos creados, sin conflictos de namespace

Test 3b: Crear modelo y exportar

```
Crea un modelo Blender de un enemigo básico (cube con textura)
y exportalo a Assets/Models/Enemy.fbx
```

Resultado: FBX en la ubicación especificada

Test 3c: Crear prefab

```
Importa el modelo Enemy.fbx, crea un prefab llamado Enemy_Prefab
y configura un BoxCollider
```

Resultado: Prefab funcional en Assets/Prefabs/

Test Suite Automatizado (Jest)

Setup

```
npm install --save-dev jest @types/jest ts-jest
```

jest.config.js

```
module.exports = {
  preset: "ts-jest",
  testEnvironment: "node",
  roots: ["<rootDir>/tests"],
  testMatch: ["**/__tests__/**/*.ts", "**/?(*.)+(spec|test).ts"],
};
```

tests/unity.test.ts

```
import { UnityCLI } from "../src/unity-cli";
import * as fs from "fs-extra";
import * as path from "path";

describe("UnityCLI", () => {
  const projectPath = "/path/to/ZoremGame";
  const unityExe = "/path/to/Unity";
  let cli: UnityCLI;

  beforeAll(() => {
    cli = new UnityCLI(projectPath, unityExe);
  });

  test("should validate Unity project", async () => {
    const exists = await fs.pathExists(
      path.join(projectPath, "ProjectSettings")
    );
    expect(exists).toBe(true);
  });

  test("should compile project without errors", async () => {
    const result = await cli.compile();
    expect(result.success).toBe(true);
```

```
    expect(result.errors.length).toBe(0);
  });

  test("should create script successfully", async () => {
    const testPath = "Assets/Scripts/TestScript.cs";
    // Mock: write file
    const fullPath = path.join(projectPath, testPath);
    const content = "public class TestScript { }";

    await fs.ensureDir(path.dirname(fullPath));
    await fs.writeFile(fullPath, content);

    const exists = await fs.pathExists(fullPath);
    expect(exists).toBe(true);

    // Cleanup
    await fs.remove(fullPath);
  });
});
```

Ejecutar tests

```
npm test
```

Checklist de Testing Completo

Operaciones Básicas

- ☐ CreateScript - crear archivo
- ☐ DeleteScript - eliminar archivo
- ☐ GetErrors - reportar errores
- ☐ CreateModel - generar modelo
- ☐ ExportFBX - exportar a FBX

Validación de Input

- ☐ Rechaza path con ../ (seguridad)
- ☐ Rechaza archivo que ya existe
- ☐ Valida que directorio padre existe
- ☐ Valida parámetros requeridos

Integración

- ☐ MCP responde a Claude Code
- ☐ Valores por defecto funcionan
- ☐ Environment variables son leídas correctamente
- ☐ Logs reportan status correctamente

Performance

- ☐ CreateScript < 1 segundo
- ☐ Compile < 10 segundos
- ☐ CreateModel < 30 segundos
- ☐ ExportFBX < 30 segundos

Error Handling

- ☐ Unity not found → Error claro
- ☐ Blender not found → Error claro
- ☐ Proyecto no encontrado → Error claro
- ☐ Timeout en operación larga → Error con retry suggestion

Monitoreo y Logs

Ver logs de MCP (macOS/Linux)

```
# Tail de los logs
tail -f ~/.claude/logs/mcp-*.log

# O directamente en la salida de stderr del proceso
npm start 2>&1 | tee mcp.log
```

Ejemplo de log esperado

```
[Unity MCP] Server starting with config: { ... }
[Unity MCP] Project found at /Users/renat/Desktop/ZoremGame
[Unity MCP] Listening for commands
[Unity MCP] Tool called: create_script
[Unity MCP] CreateScript: Assets/Scripts/Player.cs
[Unity MCP] File created, auto-compiling...
[Unity MCP] Compilation successful
[Unity MCP] Response sent to Claude Code
```

Criterios de Aceptación

Para dar por completado el MCP

Todos estos tests deben pasar:

```
☑ Setup
  ☑ Node.js 18+ instalado
  ☑ TypeScript compila sin errores
```

```
    ☑ MCP servers se inician sin crash

  ☑ Unity MCP
    ☑ CreateScript funciona
    ☑ DeleteScript funciona
    ☑ GetErrors funciona
    ☑ Compilación funciona
    ☑ Input validation funciona

  ☑ Blender MCP
    ☑ CreateModel funciona
    ☑ ExportFBX funciona
    ☑ Input validation funciona
    ☑ Path handling seguro

  ☑ Integración
    ☑ Aparecen en Claude Code
    ☑ Claude puede llamar tools
    ☑ Resultados se reportan correctamente
    ☑ Errores se manejan elegantly

  ☑ End-to-End
    ☑ Crear script + compilar
    ☑ Crear modelo + exportar
    ☑ Full workflow: modelo → prefab
```

Debugging

Habilitar verbose logging

En `src/index.ts`, agregar:

```
console.error("[MCP] Tool called:", request.params.name);
console.error("[MCP] Arguments:", request.params.arguments);
console.error("[MCP] Result:", result);
```

Inspeccionar procesos

```
# Ver todos los procesos Node.js
ps aux | grep node

# Ver procesos de Unity
ps aux | grep Unity

# Ver procesos de Blender
ps aux | grep blender
```

```
# Matar un proceso (útil si se traba)
kill -9 <PID>
```

Próximo: Referencia Técnica

Ve a **07_Referencia_Tecnica.md** para detalles técnicos de implementación.